College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 1 Report: Basics of Programming with Python

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab introduces the Python tools used for image processing: OpenCV, Pillow (PIL), NumPy, Matplotlib and scikit-image. It covers loading, displaying and saving images, and shows that a digital image is stored as a NumPy array of pixel intensities that can be inspected and changed directly (Outcome #1).

2. Environment and Setup

The notebook ran in Google Colab, used as a remote Jupyter kernel from VS Code. The required libraries were installed and the imports were checked:

```
# Most of these are pre-installed on Colab, but just in case:
!pip install -q opencv-python-headless scikit-image Pillow matplotlib numpy
```

```
# Verify all imports work
import cv2
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from PIL import Image
from skimage import data, io

print(f"OpenCV version:     {cv2.__version__}")
print(f"NumPy version:      {np.__version__}")
print("All libraries imported successfully!")
```

Output:

```
OpenCV version:     4.14.0
NumPy version:      2.1.3
All libraries imported successfully!
```

3. Images Used

- **cameraman.tif** – 512×512 grayscale image from scikit-image's built-in data (`data.camera()`).
- **lena_gray_256.tif** – the standard Lena test image, downloaded, converted to grayscale and resized to 256×256.

Both are saved to the `images/` folder so the later tasks can load them from disk:

```
import os

os.makedirs('images', exist_ok=True)

# ── cameraman.tif ──
# Available directly from scikit-image's built-in sample dataset
from skimage import data as skdata
from skimage.io import imsave

cameraman = skdata.camera()  # 512x512 grayscale, dtype=uint8
imsave('images/cameraman.tif', cameraman)
print(f"Saved cameraman.tif  — shape: {cameraman.shape}, dtype: {cameraman.dtype}")

# ── lena_gray_256.tif ──
# Download the classic Lena image from a public source and convert
# to 256×256 grayscale .tif to match the lab specification.
!curl -L -s -o /tmp/lena_color.png \
  "https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png"

from PIL import Image as PILImage

lena_pil = PILImage.open('/tmp/lena_color.png').convert('L').resize((256, 256))
lena_pil.save('images/lena_gray_256.tif')
print(f"Saved lena_gray_256.tif  — size: {lena_pil.size}, mode: {lena_pil.mode}")

# Verify both files exist
for f in ['images/cameraman.tif', 'images/lena_gray_256.tif']:
    print(f"  ✓ {f}  ({os.path.getsize(f):,} bytes)")
```

Output:

```
Saved cameraman.tif  — shape: (512, 512), dtype: uint8
Saved lena_gray_256.tif  — size: (256, 256), mode: L
  ✓ images/cameraman.tif  (262,400 bytes)
  ✓ images/lena_gray_256.tif  (65,658 bytes)
```

4. Task 2 – Loading and Visualizing Images

2a – Loading an image with OpenCV

`cv2.imread` returns the image as a NumPy array. OpenCV stores color channels in BGR order, but Matplotlib expects RGB, so the image is converted with `cv2.cvtColor` before it is displayed.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('images/cameraman.tif')  # Load the image data into a NumPy array

# NOTE: OpenCV loads images in BGR order, but matplotlib expects RGB.
# For a grayscale .tif loaded with default flags, OpenCV may give 3 channels (BGR).
# We convert BGR -> RGB so colours display correctly in matplotlib.
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('cameraman.tif — loaded with OpenCV')
plt.show()
```

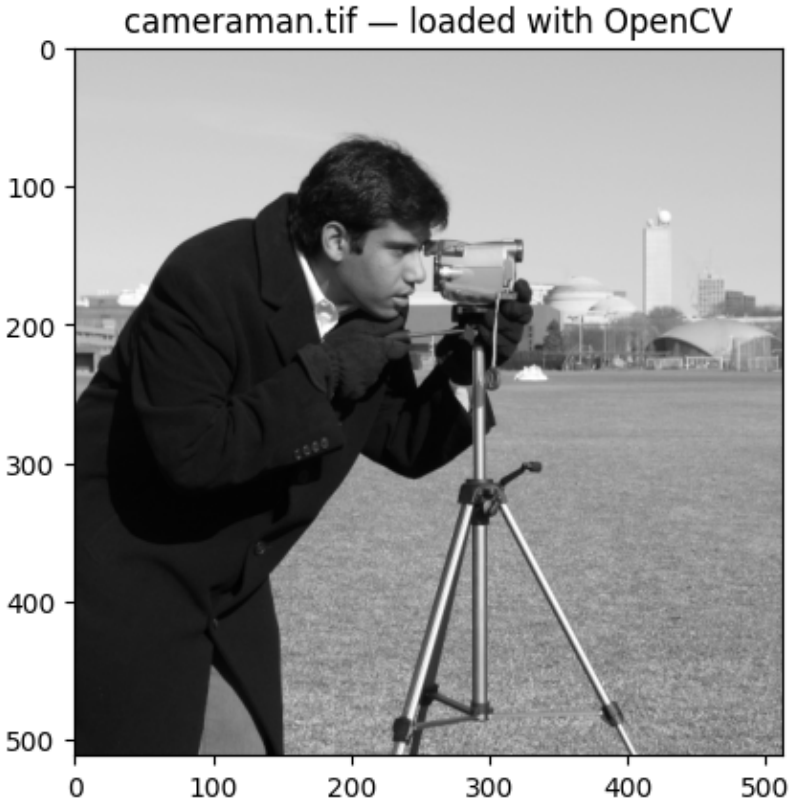


Figure 1: cameraman.tif loaded with OpenCV.

2b – Loading an image with PIL

`Image.open` returns a PIL Image object. It is shown with the reversed gray colormap (`Greys_r`) so that 0 is black and 255 is white.

```
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.cm as cm

img2 = Image.open('images/lena_gray_256.tif')  # Load image into a PIL Image object

plt.imshow(img2, cmap=cm.Greys_r)
plt.title('lena_gray_256.tif — loaded with PIL')
plt.show()
```

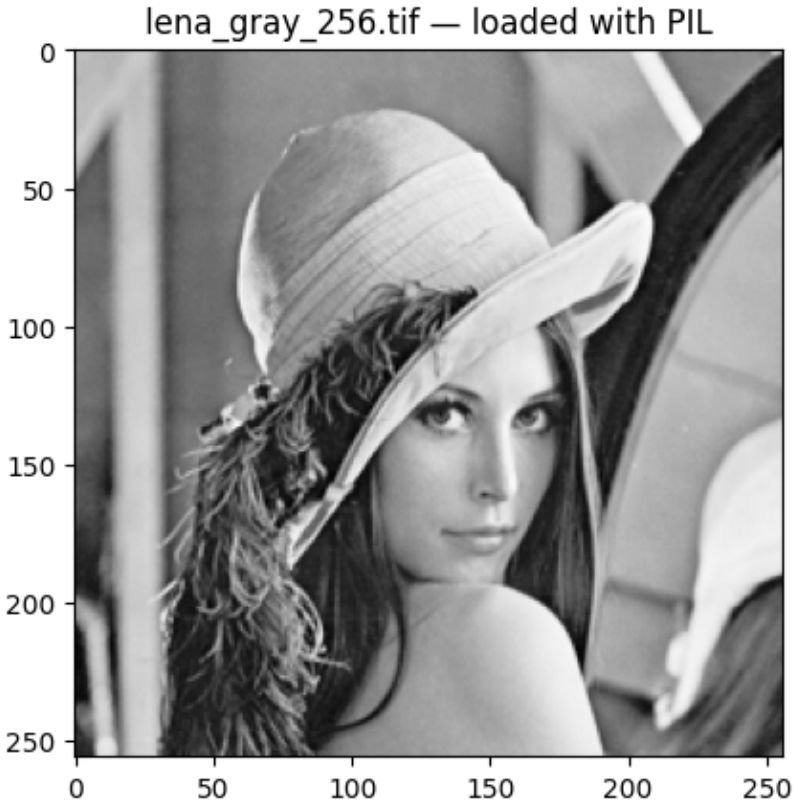


Figure 2: lena_gray_256.tif loaded with PIL.

5. Task 3 – Image Storing

Both images were written back to disk as JPEG files: the OpenCV array with `cv2.imwrite`, and the PIL image with `Image.save`.

```
# Save the OpenCV image as a JPEG
cv2.imwrite('new_image.jpg', img)
print(f"Saved 'new_image.jpg' — size: {os.path.getsize('new_image.jpg'):,} bytes")
```

Output:

```
Saved 'new_image.jpg' — size: 86,612 bytes
```

```
# Save the PIL image as a JPEG
img2.save('new_image2.jpg')
print(f"Saved 'new_image2.jpg' — size: {os.path.getsize('new_image2.jpg'):,}
bytes")
```

Output:

```
Saved 'new_image2.jpg' — size: 10,243 bytes
```

Discussion: The cameraman file (≈86.6 KB) is much larger than the Lena file (≈10.2 KB). The cameraman image has four times as many pixels (512×512 vs 256×256), and OpenCV saved it with three identical channels.

6. Task 4 – Displaying an Image as an Array

4a – OpenCV image

```
# img was loaded using OpenCV — it is already a NumPy array
print("Shape of img (OpenCV):", img.shape)
print("Data type:", img.dtype)
print()
print(img)
```

Output:

```
Shape of img (OpenCV): (512, 512, 3)
Data type: uint8

[[[200 200 200]
  [200 200 200]
  [200 200 200]
  ...
  [189 189 189]
  [190 190 190]
  [190 190 190]]

 [[200 200 200]
...
```

4b – PIL image converted to a NumPy array

```
import numpy as np

# Convert the PIL Image to a NumPy array
img_array = np.array(img2)

print("Shape of img_array (PIL → NumPy):", img_array.shape)
print("Data type:", img_array.dtype)
print()
print(img_array)
```

Output:

```
Shape of img_array (PIL → NumPy): (256, 256)
Data type: uint8

[[162 162 160 ... 170 170 143]
 [162 162 160 ... 171 172 146]
 [163 160 160 ... 159 149 116]
 ...
 [ 51  51  52 ...  82  93  92]
 [ 46  49  50 ...  93 101  99]
 [ 45  53  50 ... 100 103 106]]
```

Discussion: By default OpenCV loads every image as a 3-channel BGR array, so the grayscale cameraman image has shape (512, 512, 3), with the same value repeated in all three channels. The PIL image converted with `np.array` keeps its single channel and has shape (256, 256). Both use the `uint8` data type, so every pixel is an integer from 0 to 255.

7. Assessment – NumPy Array Operations

Array properties

```
# ── Basic array information ──
print("=== cameraman (OpenCV) ===")
print(f"  Shape      : {img.shape}")
print(f"  Dimensions : {img.ndim}")
print(f"  Size       : {img.size} pixels")
print(f"  Dtype      : {img.dtype}")
print(f"  Min / Max  : {img.min()} / {img.max()}")
print()
print("=== lena_gray_256 (PIL → NumPy) ===")
print(f"  Shape      : {img_array.shape}")
print(f"  Dimensions : {img_array.ndim}")
print(f"  Size       : {img_array.size} pixels")
print(f"  Dtype      : {img_array.dtype}")
print(f"  Min / Max  : {img_array.min()} / {img_array.max()}")
```

Output:

```
=== cameraman (OpenCV) ===
  Shape      : (512, 512, 3)
  Dimensions : 3
  Size       : 786432 pixels
  Dtype      : uint8
  Min / Max  : 0 / 255

=== lena_gray_256 (PIL → NumPy) ===
  Shape      : (256, 256)
  Dimensions : 2
  Size       : 65536 pixels
  Dtype      : uint8
  Min / Max  : 23 / 240
```

Slicing a region of interest

Array slicing crops the image directly: rows 50–199 and columns 50–199 give a 150×150 region around the face.

```
# ── Slicing: extract a region of interest ──
roi = img_array[50:200, 50:200]   # rows 50-199, cols 50-199
print(f"ROI shape: {roi.shape}")

plt.imshow(roi, cmap='gray')
plt.title('Region of Interest from lena_gray_256')
plt.show()
```

Output:

```
ROI shape: (150, 150)
```

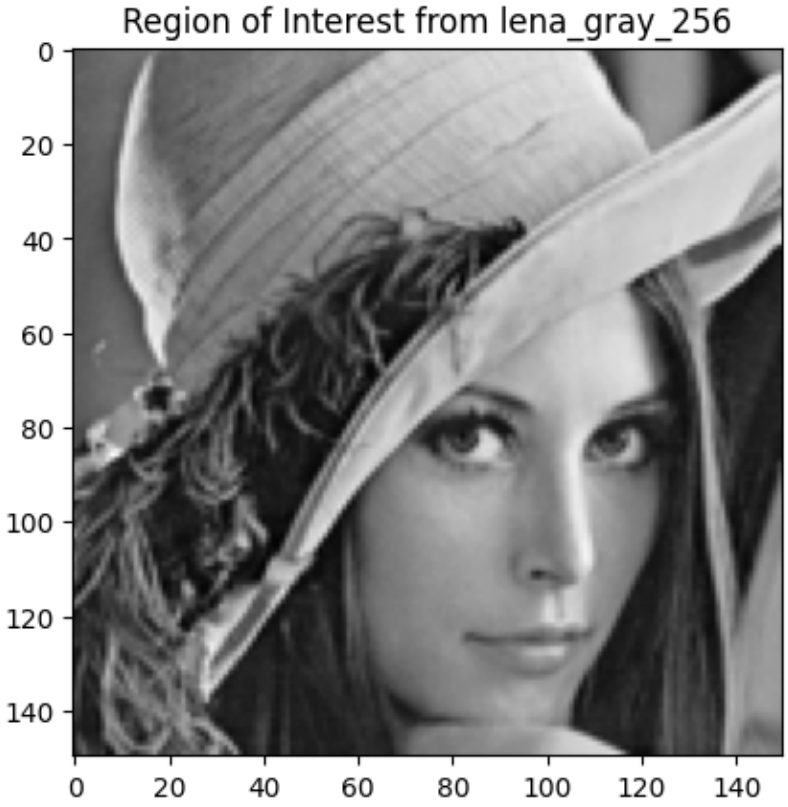


Figure 3: 150×150 region of interest cropped from lena_gray_256.

Brightness adjustment

A constant of 50 is added or subtracted. The array is first converted to `int16` and then clipped to [0, 255], so that values do not wrap around when they go past the `uint8` limits.

```
# ── Arithmetic: brightness adjustment ──
brighter = np.clip(img_array.astype(np.int16) + 50, 0, 255).astype(np.uint8)
darker   = np.clip(img_array.astype(np.int16) - 50, 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, data, title in zip(axes,
                           [darker, img_array, brighter],
                           ['Darker (−50)', 'Original', 'Brighter (+50)']):
    ax.imshow(data, cmap='gray', vmin=0, vmax=255)
    ax.set_title(title)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Figure 4: Image darkened by 50, original, and brightened by 50.

Flipping and transposing

```
# ── Flipping & transposing ──
flipped_h = np.fliplr(img_array)    # horizontal flip
flipped_v = np.flipud(img_array)    # vertical flip
transposed = img_array.T            # transpose

fig, axes = plt.subplots(1, 4, figsize=(14, 3.5))
for ax, data, title in zip(axes,
                           [img_array, flipped_h, flipped_v, transposed],
                           ['Original', 'Horizontal Flip', 'Vertical Flip',
'Transpose']):
    ax.imshow(data, cmap='gray')
    ax.set_title(title)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

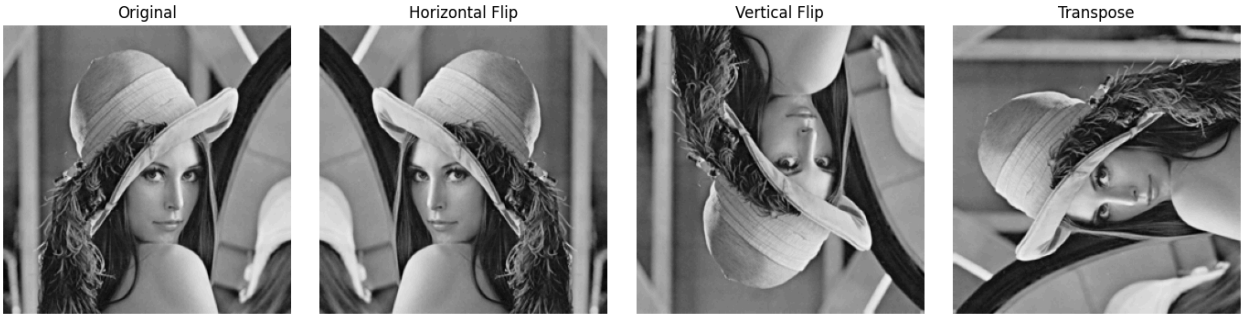


Figure 5: Original, horizontal flip, vertical flip and transpose.

Discussion: `np.fliplr` mirrors the image left to right and `np.flipud` turns it upside down. The transpose swaps rows and columns, which reflects the image across its main diagonal.

Statistical summary

```
# ── Statistical summary ──
print(f"Mean pixel value : {img_array.mean():.2f}")
print(f"Std deviation    : {img_array.std():.2f}")
print(f"Median           : {np.median(img_array):.1f}")
```

Output:

```
Mean pixel value : 124.05
Std deviation    : 47.45
Median           : 129.0
```

Discussion: The Lena image has a mean of 124.05 and a median of 129, close to the middle of the 0–255 range, and a standard deviation of 47.45. This matches its well-balanced mid-tone appearance. Its range is 23–240, so it does not use pure black or pure white.

8. Conclusion

Images were loaded with OpenCV and PIL, displayed with Matplotlib and saved to disk. The two libraries differ in how they load images: OpenCV gives a 3-channel BGR array, while PIL keeps the original mode. Once an image is a NumPy array, cropping, brightness changes, flipping and statistics are all simple array operations.